

As you can imagine, for any website or application you develop, there would be a long list of middleware you might need to create or use. Some of them might be needed to run together for a route. In this case, its better to group them under a single key to reduce your lines of code. Laravel comes with web and api middleware groups that contain common middleware you may want to apply to your web UI and API routes:

```
protected $middlewareGroups = [
    'web' => [
        \App\Http\Middleware\EncryptCookies::class,
        \Illuminate\Cookie\Middleware\AddQueuedCookies
ToResponse::class,
        \Illuminate\Session\Middleware\StartSession::class,
        \Illuminate\View\Middleware\
ShareErrorsFromSession::class,
        \App\Http\Middleware\VerifyCsrfToken::class,
        \Illuminate\Routing\Middleware\
SubstituteBindings::class,
    ],
    'api' => [
        'throttle:60,1',
        'auth:api',
    ],
];
```

Now just the same syntax as the individual middleware to assign the group to a route.

CSRF Protection

Cross-site request forgery (CSRF) attacks also known as one-click attack or session riding is a cyber attack on the application from a user that the application trusts. In this, unauthorized commands are transmitted from the user, even without their knowledge. These attacks can come in the form of specially-crafted image tags, hidden forms, and JavaScript XML[HttpRequests](#) etc. This obviously is a huge security issue.

Laravel provides built-in security to protect your application from CSRF attacks. It automatically generates a “token” for each active user session managed by the application which is used to verify that the authenticated user is the one actually making the requests to the application. To implement this, you should include a hidden CSRF token field in all of your form pages so that the CSRF protection middleware can validate the request.